Technical Project Manager Assignment

AI Meeting Assistant: 12-Week MVP Delivery Plan

Prepared for: PanScience Innovations (PSI)
Prepared by: Ajay Jha
Date: 29 October 2026

Part	Contents
1	Project Charter
2	Work Breakdown Structure
3	Delivery Roadmap
4	Sprint Planning (Sprints 1 and 2)
5	Risk Register
6	Stakeholder Communication Plan
7	Resource Plan
8	Project Dashboard Mock-up
9	Change Management Scenario
10	High-Level Architecture
11	AI Productivity Utility
App.	Methodology notes and submission checklist

Assumptions about this document. Budget figures, velocities and dashboard values are illustrative because no real data was provided. Every assumption is labelled so it can be replaced with actuals.

Contents

Contents 2

Part 1: Project Charter 3

Part 2: Work Breakdown Structure (WBS) 5

Part 3: Delivery Roadmap 6

Part 4: Sprint Planning (Sprints 1 and 2) 7

Part 5: Risk Register 9

Part 6: Stakeholder Communication Plan 10

Part 7: Resource Plan 11

Part 8: Project Dashboard Mock-up 12

Part 9: Change Management Scenario 13

Part 10: High-Level System Architecture 15

Part 11: AI Productivity and Automation Exercise 17

Appendix A: Methodology Notes and Submission Checklist 18

Appendix B: Source Code of the AI Productivity Utility (app.py) 19

Part 1: Project Charter

1.1 Project Objectives

- Deliver a production-ready MVP of **AI Meeting Assistant** in 12 weeks: it records online meetings, transcribes them, identifies speakers, produces AI summaries and action items, and makes past meetings searchable.
- Integrate with Google Calendar (auto-join scheduled meetings), Slack (share summaries) and Jira (create tickets from action items).
- Establish a scalable, secure cloud foundation so post-MVP features (multi-language, CRM) can be added without re-architecture.

1.2 Business Goals

- Validate product-market fit with 5 to 10 pilot organizations within 4 weeks of launch.
- Reduce the time teams spend on meeting follow-up (writing notes, creating tickets) by at least 50% for pilot users.
- Create a differentiated AI product line for PSI with a repeatable delivery model for future AI products.
- Keep delivery inside the 12-week window and the approved budget, with predictable reporting to leadership.

1.3 Project Scope

In scope (MVP)

- Meeting recording through a meeting bot for Google Meet, Zoom and Microsoft Teams links found on the user's calendar.
- Transcription (English) with timestamps; speaker identification (diarization) with manual speaker renaming.
- AI summary, key decisions and action items with owner and due date extraction.
- Keyword and semantic search across historical meetings; meeting library with filters.
- Google Calendar sync (read events, auto-schedule bot), Slack (post summary to a channel, DM action items), Jira (create and link issues from action items).
- User authentication (email plus Google SSO), organization workspaces, role-based access (admin, member).
- Web application (responsive), basic admin settings, usage analytics for pilots.
- Security baseline: encryption in transit and at rest, audit logs, consent notification when recording.

Out of scope (MVP)

- Native mobile apps and a desktop recorder; in-person meeting recording.
- Multi-language transcription/summaries, voice cloning, CRM integrations (candidates for post-MVP).
- Custom model training or fine-tuning; on-premise deployment.
- Billing/subscription management (pilots are free or invoiced manually); SOC 2 certification (controls prepared, audit later).
- Integrations beyond Google Calendar, Slack and Jira (for example Outlook Calendar, Teams chat, Asana).

1.4 Assumptions

- The team listed in the brief is fully available (see Resource Plan for planned leave) from Week 1.
- A managed meeting-bot API vendor and a managed speech-to-text and LLM provider will be used for the MVP instead of building these from scratch.
- The Product Manager provides a signed-off PRD by end of Week 1 and design approvals within 2 business days.
- Cloud account (AWS), third-party developer accounts (Google, Slack, Atlassian) and API budgets are provisioned in Week 1.
- Pilot customers accept an English-only MVP and will provide recording consent.

1.5 Constraints

- **Time:** fixed 12-week deadline. **Team:** 12 people, no additional hiring approved except short-term contractors (see Part 7).
- **Compliance:** recordings contain personal data, so consent, retention and data-residency rules must be respected (GDPR-style).
- **Third-party limits:** OAuth app reviews (Google, Slack, Atlassian) and API rate limits are outside team control.
- **AI cost:** ASR and LLM usage cost scales with meeting minutes, so a per-meeting cost cap is required.

1.6 Success Metrics

Category	Metric	Target at MVP launch
Delivery	MVP live by Week 12; sprint commitment reliability	On time; at least 85% of committed points delivered per sprint
Quality	Critical / high bugs open at launch	0 critical, 0 high; escaped defects under 5 in first 2 weeks
AI quality	Transcript word error rate (English, clean audio)	At most 12%
AI quality	Speaker attribution accuracy	At least 85%
AI quality	Action item recall / precision (human-rated sample of 50 meetings)	At least 75% / at least 80%
Performance	Summary ready after a 60-minute meeting ends	Within 10 minutes (p90)
Reliability	Recording success rate; API availability	At least 97%; 99.5%
Adoption	Pilot orgs onboarded; weekly active users among invited	5 orgs; at least 40%
Satisfaction	Pilot NPS / summary usefulness rating	NPS at least 30; 4 out of 5

1.7 Major Deliverables

- Signed-off PRD, design system and high-fidelity UI designs; architecture and API specifications.
- Production cloud environment with CI/CD, monitoring and runbooks.
- Core product: recording, transcription, speaker ID, summary and action items, search, web app.
- Integrations: Google Calendar, Slack, Jira.
- Test assets: automated regression suite, performance and security test reports, UAT sign-off.
- Launch package: user guide, release notes, support playbook, post-launch hypercare plan.

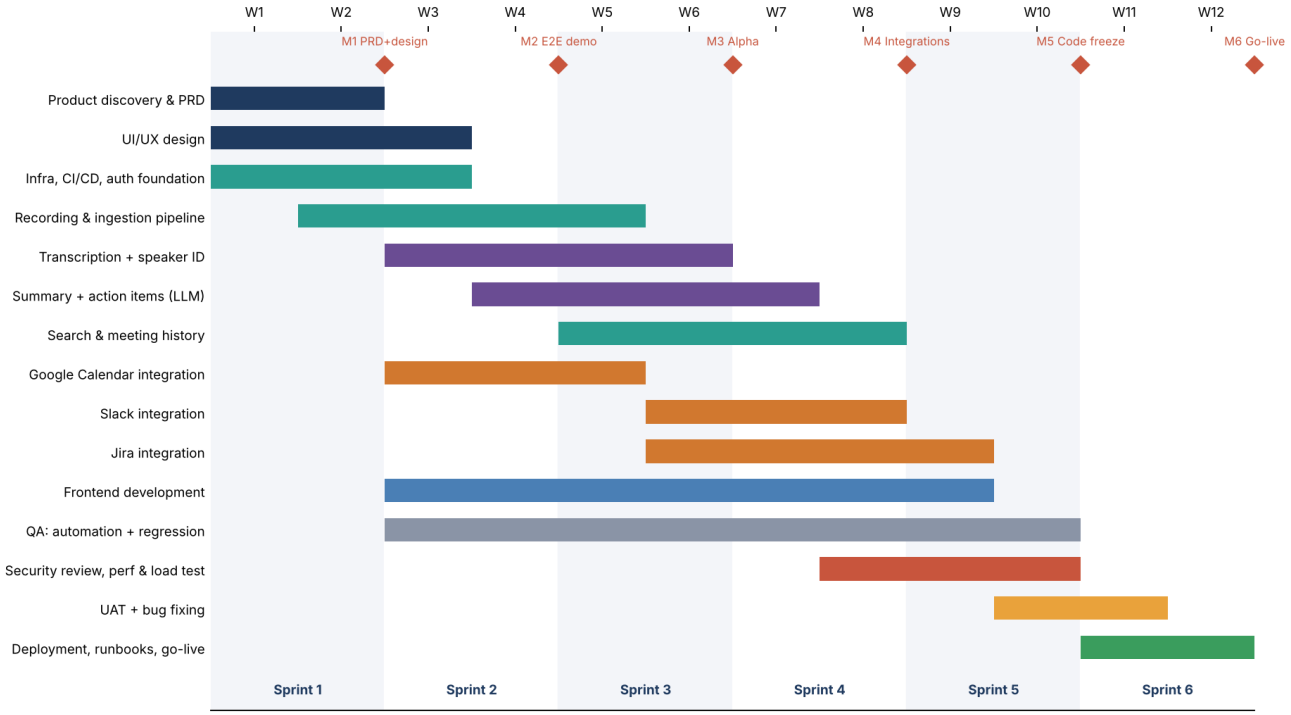
Part 2: Work Breakdown Structure (WBS)

The WBS is organized into 10 workstreams, decomposed to work-package level (3 levels). IDs are reused in the roadmap, sprint plan and dashboard. Estimates are in person-weeks (pw).

ID	Workstream / Work package	Owner	Effort (pw)
1.0	**Product Discovery**	PM	3
1.1	Stakeholder interviews, personas, competitor scan	PM	
1.2	Success metrics and MVP scope lock	PM + TPM	
2.0	**Requirement Finalization**	PM	3
2.1	PRD, epics and user stories, backlog grooming	PM	
2.2	Non-functional requirements (security, privacy, performance)	TPM + Eng	
2.3	Definition of Done and acceptance criteria standards	TPM + QA	
3.0	**UI/UX Design**	Designer	8
3.1	User flows and wireframes	Designer	
3.2	Design system and high-fidelity screens	Designer	
3.3	Usability test with 5 users; design handoff	Designer + PM	
4.0	**Backend Development**	Backend Lead	26
4.1	Cloud foundation, CI/CD, environments	Backend + DevOps	
4.2	Auth, organizations, RBAC	Backend	
4.3	Meeting service and calendar sync	Backend	
4.4	Recording ingestion, storage, job queue	Backend	
4.5	Search service (keyword + semantic)	Backend	
4.6	Integration service (Slack, Jira)	Backend	
5.0	**AI Model Integration**	AI/ML Lead	16
5.1	ASR vendor benchmark and integration	AI/ML	
5.2	Speaker diarization	AI/ML	
5.3	Summary and action item prompts, evaluation set	AI/ML	
5.4	Embeddings, retrieval quality tuning	AI/ML	
5.5	Cost and latency optimization, guardrails	AI/ML	
6.0	**Frontend Development**	Frontend Lead	14
6.1	App shell, auth, navigation	Frontend	
6.2	Meeting library, transcript viewer, summary view	Frontend	
6.3	Search UI, settings, integrations UI	Frontend	
7.0	**QA and Testing**	QA Lead	14
7.1	Test strategy, test data, automation framework	QA	
7.2	Functional, regression, integration tests	QA	
7.3	Performance, load, security testing; AI quality evaluation	QA + AI/ML	
8.0	**Deployment**	Backend + DevOps	4
8.1	Production environment, monitoring, alerting, backups	DevOps	
8.2	Release pipeline, rollback plan, runbooks	DevOps	
9.0	**User Acceptance Testing**	PM + QA	3
9.1	UAT plan, pilot users, feedback triage	PM	
9.2	Fix and retest cycle, sign-off	QA	
10.0	**Production Launch**	TPM	2
10.1	Go/no-go review, launch communication	TPM	
10.2	Hypercare (2 weeks), launch retrospective	TPM + Eng	
11.0	**Project Management** (continuous)	TPM	12
11.1	Planning, ceremonies, reporting, risk and change control	TPM	

Part 3: Delivery Roadmap

Methodology: Scrum with 2-week sprints (6 sprints), a 3-day pre-sprint discovery overlap in Week 1, and phase gates for design approval, code freeze and go-live. Releases: Internal Alpha (Week 6), Closed Beta with pilot customers (Week 9), Release Candidate (Week 10), MVP GA (Week 12).



3.1 Phases and Sprint Schedule

Sprint	Weeks	Phase	Focus	Exit milestone
S1	1-2	Discovery, design, foundation	PRD lock, design system, infra, auth, vendor spikes (ASR, bot)	M1: PRD and design approved
S2	3-4	Core build	Calendar sync, recording ingestion, transcript pipeline v1, UI shell	M2: record to transcript E2E demo
S3	5-6	Core build and AI	Speaker ID, summary and action items, library UI, Slack start	M3: Internal Alpha (Week 6)
S4	7-8	Integrations and search	Search, Slack, Jira, settings, performance baseline	M4: Integrations feature complete
S5	9-10	Hardening	Closed Beta, bug fixing, security test, load test, AI tuning, code freeze (end Week 10)	M5: Release Candidate, code freeze
S6	11-12	UAT and launch	UAT fixes, deployment, runbooks, go-live, hypercare start	M6: MVP GA (end Week 12)

3.2 Key Dependencies

Dependency	Needed by	Blocks
PRD and design approval (PM, Designer)	End Week 2	Frontend build, story refinement for S3
Cloud accounts, API keys, OAuth app registrations (Google, Slack, Atlassian)	Week 1-2	Calendar, Slack and Jira integration; Google review can take 2 to 4 weeks, so submit in Week 1
Meeting-bot vendor contract and sandbox	Week 2	Recording pipeline
ASR benchmark decision	End Week 3	Transcription, speaker ID, cost model
Recording to transcript pipeline stable	End Week 4	Summary, action items, search
Summary quality evaluation set (50 labelled meetings)	Week 5	Alpha exit criteria, prompt tuning
Code freeze and QA sign-off	End Week 10	UAT, deployment

Part 4: Sprint Planning (Sprints 1 and 2)

Capacity basis: 10 delivery engineers (4 backend, 2 frontend, 2 AI/ML, 2 QA) plus 1 designer. 10 working days per sprint, 80% focus factor. Sprint 1 is planned at 50 points (team ramp-up) and Sprint 2 at 58. 1 story point is roughly half a day of ideal effort for one engineer; priorities use MoSCoW (Must, Should, Could).

Sprint 1 (Weeks 1-2): Foundation and Design

Sprint Goal: Lock the scope and design, stand up the cloud and CI/CD foundation with working authentication, and de-risk the two biggest technical unknowns (ASR quality and the meeting bot).

Deliverables: approved PRD and backlog; design system and hi-fi screens for core flows; dev and staging environments with CI/CD; login with Google SSO; ASR and bot vendor benchmark report; test strategy.

ID	User story	Pri	SP	Acceptance criteria
US-101	As a PM, I want a finalized PRD and prioritized backlog so the team builds the right thing.	Must	3	PRD signed by PM and engineering lead; top 40 stories have acceptance criteria; Definition of Done agreed
US-102	As a user, I want clear UI flows and a design system so the product feels consistent.	Must	8	Wireframes for onboarding, calendar connect, meeting library, transcript view; hi-fi screens approved within 2 days of review; component library in Figma
US-103	As an engineer, I want CI/CD and dev/staging environments so code ships safely.	Must	8	Pipeline runs lint, unit tests and deploys to staging on merge; infrastructure as code; rollback documented
US-104	As a user, I want to sign up and log in with email or Google so I can access my workspace.	Must	8	Sign up, login, logout, password reset work; Google SSO works; sessions expire per policy; unit and API tests pass
US-105	As an admin, I want an organization workspace with roles so I control access.	Should	5	Org created at signup; admin and member roles enforced on API; invite by email works
US-106	As a product team, we want an ASR benchmark so we pick a vendor on evidence.	Must	5	3 vendors tested on 20 recordings; WER, latency and cost per hour compared; recommendation documented
US-107	As a product team, we want a meeting-bot proof of concept so we know joining meetings is feasible.	Must	5	Bot joins a Google Meet and a Zoom test meeting and returns audio file; failure cases noted
US-108	As a QA, I want a test strategy and automation skeleton so testing starts early.	Should	3	Test plan approved; API and UI automation framework runs in CI; test data set defined
US-109	As a user, I want a basic app shell with navigation after login so features can plug in.	Should	5	Responsive layout, routing, login screen integrated with auth API

Total committed: 50 points (3+8+8+8+5+5+5+3+5). Must-haves = 37; Should = 13 can slip without affecting the goal. Sprint buffer: 10% held for unplanned work, so commitment is trimmed by removing US-109 if US-106 or US-107 spikes overrun.

Sprint 2 (Weeks 3-4): Core Pipeline

Sprint Goal: Demonstrate an end-to-end flow where a scheduled calendar meeting is joined, recorded and turned into a viewable transcript in the web app (Milestone M2).

Deliverables: Google Calendar connect and sync; meeting service; recording ingestion with storage and job queue; transcript v1 shown in UI; meeting library page; E2E demo; automated tests for the pipeline.

ID	User story	Pri	SP	Acceptance criteria
US-201	As a user, I want to connect my Google Calendar so my meetings appear in the app.	Must	8	OAuth consent flow works; upcoming 14 days of events with meeting links synced; re-sync every 5 minutes; disconnect removes tokens
US-202	As a user, I want the assistant to auto-join my selected meetings so I do not have to start recording.	Must	8	Toggle per meeting and a default; bot joins within 1 minute of start; consent notice posted in the meeting; 90% join success in test set
US-203	As a system, I need secure recording ingestion and storage so audio is processed reliably.	Must	8	Audio stored encrypted in object store; job created per meeting; retries with dead-letter queue; failed jobs visible to ops
US-204	As a user, I want a transcript with timestamps so I can read what was said.	Must	8	Transcript produced in under 0.5x meeting duration; timestamps per utterance; WER target measured on benchmark set
US-205	As a user, I want a meeting library so I can find and open my past meetings.	Must	5	List with title, date, duration, status; pagination; opens transcript page; empty and error states handled
US-206	As a user, I want a transcript viewer so I can read, scroll and click timestamps to play audio.	Should	8	Transcript view with audio player sync; loading, failed and processing states; works on tablet width
US-207	As a team, we want a speaker diarization spike so we know how to label speakers.	Should	5	Two approaches tested; accuracy on 10 meetings reported; recommendation for Sprint 3
US-208	As a QA, I want automated E2E and API tests for the pipeline so regressions are caught.	Must	5	E2E test from calendar event to transcript runs nightly in staging; pass rate reported on dashboard

ID	User story	Pri	SP	Acceptance criteria
US-209	As an admin, I want basic monitoring and alerts so I know when processing fails.	Should	3	Dashboards for queue depth and error rate; alert to Slack on job failure rate above 5%

Total committed: 58 points. Must = 42; Should = 16. The Should stories (US-206, US-207, US-209) are the planned trade-off pool. Definition of Done for all stories: code reviewed, unit tests passing, deployed to staging, acceptance criteria verified by QA, documentation updated.

Part 5: Risk Register

Scoring: Probability (P) and Impact (I) each rated 1 to 5. Severity = P x I. High: 15 or more; Medium: 8 to 14; Low: below 8. The register is reviewed weekly and the top 5 are reported to leadership.

ID	Risk description	P	I	Sev.	Mitigation plan	Owner
R1	AI accuracy is below target: transcription errors, wrong speakers, or poor action item extraction reduce trust.	4	4	16 High	Vendor benchmark in S1; labelled evaluation set of 50 meetings; weekly quality score; human-edit UI for speakers; fallback to best-in-class vendor; set expectations on accuracy in beta	AI/ML Lead
R2	Third-party API failures or limits (meeting bot, Google, Slack, Jira, LLM) cause missed recordings or outages.	3	5	15 High	Queue with retry and back-off; circuit breakers; vendor SLAs; sandbox tests in S1; status monitoring; secondary LLM provider	Backend Lead
R3	Scope creep from stakeholder requests delays the MVP.	4	4	16 High	Change control process (Part 9); scope baseline signed in Week 1; prioritized backlog with Should/Could pool as buffer; weekly scope review	TPM
R4	Privacy and security: recordings hold sensitive data; vulnerabilities or consent gaps create legal exposure.	3	5	15 High	Threat model in S4; encryption and RBAC from S1; consent notice; data retention settings; third-party penetration test in S5; no training on customer data	Backend Lead
R5	Delayed design approvals block frontend work.	3	3	9 Med	Design SLA of 2 business days; approvals at fixed weekly review; frontend starts from design system and wireframes before final polish	Product Manager
R6	Google/Slack/Atlassian OAuth app review delays integrations.	3	4	12 Med	Submit apps in Week 1; use test mode and allow-listed pilot users; keep integration scope minimal in permissions	Backend Lead
R7	Infrastructure bottlenecks and cost: GPU or API quotas, slow processing, or runaway AI cost.	3	4	12 Med	Quota requests in Week 1; autoscaling workers; per-meeting cost cap and dashboard; load test with 100 concurrent meetings in S5	Backend Lead
R8	Resource availability: attrition, illness or leave of key engineers (single-person dependencies such as AI/ML).	3	4	12 Med	Pair on critical modules; documentation; cross-training; contractor option; leave calendar in Week 1	TPM
R9	Performance issues: long meetings produce slow transcripts, summaries or search.	3	3	9 Med	Chunked processing; performance budget in acceptance criteria; load tests in S4 and S5; caching of embeddings	AI/ML Lead
R10	Integration failures or data mismatch between modules found late.	3	4	12 Med	Contract-first API design; E2E tests from S2; integration test environment; demo every sprint	QA Lead
R11	Production deployment risks: failed release, misconfiguration, no rollback.	2	5	10 Med	Infrastructure as code; staging parity; blue/green deploy; rollback rehearsal in Week 10; release checklist and go/no-go review	DevOps / Backend Lead
R12	UAT finds major gaps late, compressing the fix window.	3	4	12 Med	Closed beta in Week 9; early demos in S3 and S4; reserve 20% of S5 and S6 capacity for fixes	Product Manager
R13	Meeting platform policies (Zoom, Teams) restrict bots or require approvals.	2	4	8 Med	Use vendor with approved apps; test all three platforms in S1; prioritize Google Meet and Zoom if blocked	Product Manager

Part 6: Stakeholder Communication Plan

Stakeholder	Frequency	Meeting type	Reports	Channel	Escalation process
Leadership (CEO, CTO, Head of Product)	Weekly update; monthly steering committee; milestone reviews	30-min steering review; milestone demo	One-page weekly status (RAG), dashboard, risk top 5, decisions needed, monthly budget view	Email + Slack #psi-ai-meeting-exec; dashboard link	Red items escalate to the CTO within 24 hours; scope or date impact above 1 week goes to steering committee
Product Team (PM, stakeholders, pilot customers)	Twice weekly sync; sprint review and planning every 2 weeks	Backlog grooming; sprint review demo; beta feedback sessions	Sprint review notes, backlog health, release notes, beta feedback log	Slack #product; Jira; Notion	Unresolved priority conflicts go from PM to TPM, then to Head of Product within 2 days
Engineering (backend, frontend, AI/ML)	Daily 15-min stand-up; weekly tech sync; sprint retrospective	Stand-up; architecture review; code review rhythm; demo	Burndown, velocity, blockers list, tech debt log	Slack #eng; Jira; GitHub	Blocker not resolved within 1 day goes to Engineering Lead, then TPM; critical issues paged immediately
Design (UI/UX)	Weekly design review; ad hoc pairing	Design critique; handoff session; usability test	Design status and approval tracker	Figma comments; Slack #design	Approval delay above 2 days is flagged by TPM to PM; above 4 days to Head of Product
QA	Daily in stand-up; weekly quality review; bug triage twice weekly	Test planning; bug triage; release readiness review	Test execution report, open bugs by severity, automation pass rate, quality gates	Slack #qa; Jira	Critical bug triggers same-day triage and TPM notification; release readiness reviewed with Engineering Lead
External vendors (meeting-bot, ASR/LLM, pen-test firm)	Weekly sync during integration; monthly business review	Vendor sync call; incident bridge	SLA and usage report, cost report, open vendor tickets	Shared Slack channel; email; vendor portal	Account manager, then vendor executive sponsor; contractual SLA breach goes to Leadership

6.1 Escalation Ladder

- **Level 1 (team):** owner and TPM resolve within 1 working day.
- **Level 2 (functional leads):** Engineering Lead or PM within 2 working days.
- **Level 3 (leadership):** CTO or Head of Product when delivery date, budget or customer commitments are at risk.
- **Incidents (S1/S2 production):** immediate page, incident channel, update every 30 minutes until resolved, post-incident review within 3 days.

6.2 Meeting Cadence Summary

Ceremony	When	Duration
Daily stand-up	Daily, 10:00	15 min
Sprint planning	Day 1 of sprint	2 hours
Backlog grooming	Mid-sprint	1 hour
Sprint review and demo	Last day	1 hour
Retrospective	Last day	45 min
Risk review	Weekly	30 min
Steering committee	Monthly + milestones	30 min

Part 7: Resource Plan

7.1 Team Allocation

Role	Count	Allocation	Primary responsibilities	Peak weeks
Product Manager	1	100%	PRD, priorities, UAT, stakeholder alignment	1-2, 9-11
Technical Project Manager (me)	1	100%	Planning, risk, reporting, change control, release	All
Backend Engineers	4	100%	BE1 platform/auth; BE2 meetings and calendar; BE3 ingestion and queue; BE4 integrations and search	3-9
Frontend Engineers	2	100%	FE1 app shell, library, transcript; FE2 search, settings, integrations UI	3-9
AI/ML Engineers	2	100%	ML1 ASR and diarization; ML2 LLM summaries, embeddings, evaluation	2-8
QA Engineers	2	100%	QA1 functional and E2E automation; QA2 integration, performance, AI evaluation	5-11
UI/UX Designer	1	100% in W1-4, then 50%	Design system, screens, usability test, then polish and UAT support	1-4

7.2 Capacity Planning

Assumptions: 10 working days per sprint, 80% focus factor (meetings, reviews, support), 1 planned leave day per sprint across the team. Capacity is expressed in story points (1 SP is about half a day of ideal effort).

Sprint	Dev + QA people	Available days	Capacity (SP)	Planned load (SP)	Notes
S1	10	~72	~45-50	50	Ramp-up; design-heavy, so some engineers work on spikes
S2	10	~76	~55-60	58	Full capacity
S3	10	~76	~58-62	55-58	Alpha crunch; buffer for rework
S4	10	~74	~58-60	55	Integrations; external dependencies
S5	10	~76	~58-62	50 + fixes	Hardening; 20% reserved for bugs
S6	10	~72	~52-56	40 + UAT fixes	Launch; reduced new features

7.3 Resource Dependencies and Critical Skills

- **AI/ML engineers are a single point of failure** for ASR, diarization and LLM prompts. ML1 and ML2 cross-review each other's work; Backend Lead is secondary on pipeline integration.
- **Designer** output gates frontend work in Weeks 1 to 4; frontend starts with a component library before screens are final.
- **QA** needs stable staging by Week 3 and labelled AI evaluation data by Week 5 from the PM and AI/ML team.
- **Critical skills:** cloud and DevOps (AWS, Kubernetes, CI/CD), real-time audio and queue-based pipelines, prompt engineering and LLM evaluation, OAuth and webhook integrations, application security.

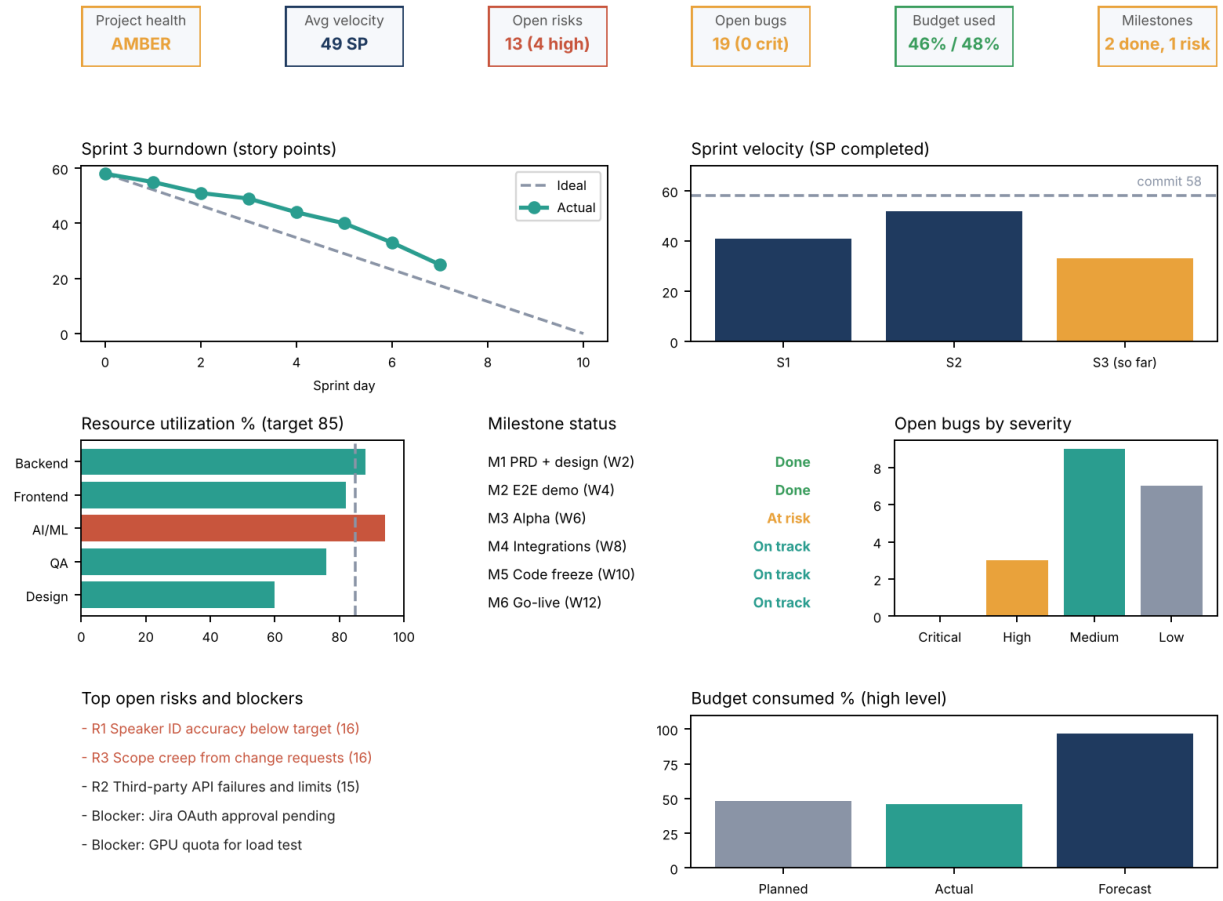
7.4 Hiring and Outsourcing Requirements

Need	Type	Duration	Reason
DevOps / SRE engineer	Contractor, 50%	Weeks 1-12	Nobody on the team is a dedicated DevOps; avoids the backend team losing about 15% capacity to infrastructure
Third-party penetration test	Vendor	Weeks 8-9 (about 1 week)	Independent security validation for sensitive recorded data
Meeting-bot API, ASR and LLM providers	Vendor (buy, not build)	Contract in Week 2	Saves 8 to 12 weeks versus building bot infrastructure and models
Technical writer / support content	Optional contractor	Weeks 10-12	User guide and support playbook so engineers stay on the product

Part 8: Project Dashboard Mock-up

The leadership dashboard shows project health at a glance and is refreshed every Friday. It would be built in Jira dashboards plus a Google Sheet for budget and utilization. Values below are mock data for the end of Sprint 3; sprint figures match the sample data used by the Part 11 utility, and risk counts match the Part 5 register.

AI Meeting Assistant | Leadership Dashboard (mock data, end of Sprint 3)



8.1 Metric Definitions and Thresholds

Metric	Definition	Green	Amber	Red
Sprint velocity	Story points completed per sprint vs committed	At least 90% of commit	70-90%	Below 70%
Burndown	Remaining points vs ideal line	On or below ideal	Up to 15% above ideal	More than 15% above
Project health	Composite of pace, risks, bugs, blockers, milestones	Score 0-1	Score 2-3	Score 4 or more
Open risks	Open risks by severity	No high	1-2 high	3 or more high
Open bugs	By severity; trend per week	No critical; falling trend	Critical fixed within 2 days	Open critical or rising trend
Milestones	Done / on track / at risk / late	All on track	One at risk	Any late
Resource utilization	Booked vs available hours	75-90%	90-100% or below 70%	Above 100%
Budget	Spent % vs planned %, with forecast at completion	Within 5%	5-10% variance	Above 10%

Part 9: Change Management Scenario

Situation: After Sprint 3 (end of Week 6), the CEO requests multi-language support, voice cloning and CRM integration. 6 weeks remain; the team is about 95% committed with a thin buffer.

9.1 Evaluate the Request

- Meet the CEO within 24 hours to understand the **why**: which customer, deal or strategic goal drives each feature, the deadline, and what 'done' means (for example 3 languages or 20; which CRM).
- Log each feature as a formal change request with business value, urgency, owner and success criteria. No work starts until it is triaged.
- Run a 2-day technical assessment with the engineering leads (not just estimates from memory) covering design, vendor options, data and compliance impact.

9.2 Assess Effort and Project Impact

Feature	What it involves	Rough effort	Key risks	Impact on MVP (Week 12)
Multi-language support	Multi-language ASR, language detection, summary and action items in other languages, UI localization, evaluation sets per language	Pilot with 2 to 3 languages: about 14 to 18 person-weeks (2 AI/ML, 1 backend, 1 frontend, QA)	Quality varies by language; need native-speaker evaluation; cost increase	High. Would consume about 35% of remaining capacity and delay beta
Voice cloning	Voice model training on user audio, synthesis, consent and misuse controls	Large: 20+ person-weeks plus legal review; no current skill on the team	Ethical, legal and security risk (deepfake, consent, biometric data law); not related to the core summarisation value	Very high. Cannot fit without cutting core scope
CRM integration	OAuth, mapping of meeting notes and action items to contacts, deals and activities for one CRM first (for example HubSpot or Salesforce)	CRM-lite (push summary to a contact or deal): about 5 to 7 person-weeks	CRM data mapping, rate limits, app approval time	Medium. Fits only if Jira or Slack scope shifts or is shortened

Capacity check: remaining capacity is about 280 SP (about 60 person-weeks at focus factor). Committed work, bug buffer and launch tasks use about 90%. The three features together would add roughly 40 to 45 person-weeks, which is not absorbable without moving the date or removing core scope.

9.3 Prioritize the Features

Scoring using RICE-style value vs effort and risk, plus alignment with the MVP goal (meeting summaries and actions):

Feature	Value	Effort	Risk	Recommendation
CRM integration (CRM-lite)	High for sales-led pilots	Medium	Low-Med	**Do first after MVP**, or include a minimal version if displacing Jira depth is acceptable
Multi-language	High for market expansion	High	Medium	**Release 1.1** (Weeks 13-18) with 2 languages; build language-ready architecture now (cheap)
Voice cloning	Unclear / low for core use	Very high	High (legal, ethics)	**Do not schedule**; run discovery and legal review, revisit with customer demand evidence

9.4 Communicate Trade-offs

- Prepare a one-page decision brief with three options and let leadership choose: **Option A** hold the 12-week date and defer all three to Release 1.1 (recommended); **Option B** add CRM-lite and trim Jira to create-only, date unchanged, with moderate risk; **Option C** include everything and move launch by about 8 weeks (to about Week 20).
- Show the cost of each option in plain terms: dates, quality risk, budget impact and which committed feature is dropped. Use the same capacity numbers that appear in the dashboard so the discussion is fact-based.
- Frame it as sequencing, not refusal: 'We can deliver all of this, in this order'. Agree explicitly on what is not being built in the MVP.

9.5 Update the Roadmap

- Re-baseline the backlog after the decision; record the change in the change log with approver and date.
- Recommended plan: MVP GA stays at Week 12; **Release 1.1** (Weeks 13-18) delivers CRM integration and multi-language (2 languages); voice cloning moves to a discovery spike in Weeks 13-14.
- Make low-cost preparations inside the MVP: store language metadata, externalize UI strings, and keep the integration service extensible. Add the new items to the risk register (R3 scope creep, new vendor and compliance risks).

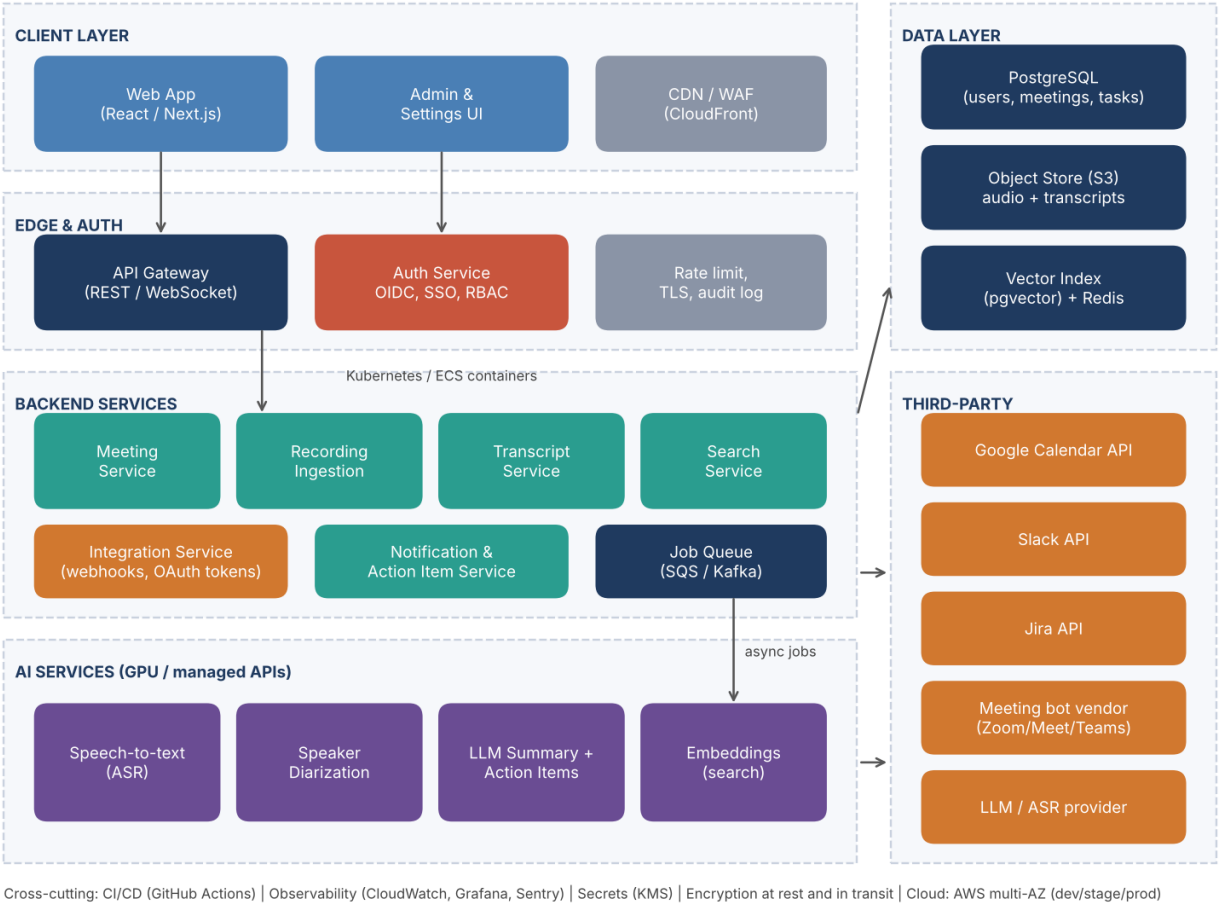
- If the CEO insists on any feature inside the MVP, swap by scope (remove equal effort) and add the corresponding reduction in Jira, Slack or search depth rather than silently overloading the team.

9.6 Manage Stakeholder Expectations

- Share the decision and the revised roadmap with all stakeholders the same day; update the dashboard so scope and dates are visible.
- Hold a short monthly roadmap review for Release 1.1 so the CEO sees progress on requested features.
- Reconfirm commitments with the team in sprint planning so they know what changed and why; protect the team from informal additions by routing all requests through the PM and TPM.

Part 10: High-Level System Architecture

The architecture is a cloud-native, event-driven design on AWS. The web app talks to an API gateway; backend microservices (modular monolith acceptable at MVP) handle meetings, ingestion, transcripts, search and integrations. Long-running work (transcription, diarization, summarization, embeddings) runs asynchronously through a job queue so the user experience is not blocked.



10.1 Component Notes and Key Decisions

Layer	Choice	Reason
Frontend	React / Next.js, responsive, hosted behind CDN and WAF	Fast development; team skills; SEO not critical
Backend	Node.js or Python services (container) on ECS/Kubernetes; REST and WebSocket for live status	Independent scaling of ingestion and API; simple deploys
AI services	Managed ASR and LLM APIs for MVP; diarization via vendor or open-source model on GPU; embeddings for semantic search	Fastest path to quality; keep abstraction layer so vendors can be swapped
Authentication	OIDC provider (for example Auth0 or Cognito), Google SSO, JWT with RBAC, organization isolation in every query	Do not build identity in-house; security by default
Database	PostgreSQL (with pgvector for search), Redis cache, S3 for audio and transcripts	Single relational store; encryption at rest; per-tenant retention policies
Cloud infrastructure	AWS multi-AZ; infrastructure as code; GitHub Actions CI/CD; blue/green releases; CloudWatch, Grafana, Sentry	Reliability, rollback, observability from day one
Integrations	Google Calendar (OAuth, push notifications), Slack (Bolt app), Jira (OAuth 2.0 Cloud API), meeting bot vendor, LLM and ASR provider	All external calls go through the Integration Service with retry, rate limiting and token storage in a secret manager

10.2 Core Data Flow

- Calendar event sync, then the bot joins the meeting at start time (with consent notice) and streams or uploads audio to object storage.
- Ingestion service writes metadata to PostgreSQL and publishes a job; ASR and diarization produce a speaker-labelled transcript.

- LLM service creates a summary, decisions and action items; embeddings are stored for search.
- Notification service posts the summary to Slack and creates Jira issues when the user approves; the web app updates through WebSocket.

Part 11: AI Productivity and Automation Exercise

Option chosen: Option C (Project Status Report Generator) with a transparent health scoring engine. **Name:** PSI Sprint Status Reporter.

11.1 Problem and Product Thinking

- **Repetitive task:** every Friday a TPM collects sprint numbers, risks, blockers and milestones from several tools and rewrites them into a leadership status report. It takes 45 to 60 minutes and is inconsistent from week to week.
- **Solution:** a command-line tool that reads a JSON export of sprint data and produces a Markdown and HTML status report with an explainable RAG health status, risk table, milestone table, blockers and decisions needed. An optional *--ai* flag adds an executive summary written by the Claude API.
- **Design choices:** rule-based health scoring (so leadership can see why a project is Amber), zero third-party dependencies (runs anywhere with Python 3), and graceful fallback when AI or network is unavailable.
- **Expected impact:** report preparation goes from about 45 minutes to about 5 minutes, and the health rules are the same every week.

11.2 AI Tools and Workflow Used

- **Claude** was used to build the utility: I described the manual task, asked for transparent scoring rules instead of a black-box score, reviewed the thresholds, and iterated on the output layout using mock Sprint 3 data.
- Claude was also used to help structure the planning documents, draft risk and communication tables, and generate the diagrams; every number was then reviewed and made consistent across parts.
- Prompts used: (1) 'Write a Python CLI that converts sprint metrics JSON into a weekly leadership status report in Markdown and HTML.' (2) 'Compute RAG health with simple explainable rules and list the reasons.' (3) 'Add an optional AI executive summary via the Anthropic API, but keep the tool working offline.'

11.3 Sample Output (from sample_data.json)

Field	Value
Health	AMBER (pace 81% of plan; 2 open high-severity risks; 1 milestone at risk)
Sprint 3 progress	33 of 58 SP completed at day 7 of 10
Velocity history	41, 52, 56 SP
Bugs	19 open (0 critical), 11 closed this week
Blockers	Jira OAuth approval pending; GPU quota for load test
Decisions needed	Approve defer of voice cloning; confirm HubSpot as first CRM

11.4 Setup Instructions (README)

```
git clone <your-repo-url>
cd psi-status-reporter
python app.py sample_data.json          # writes report.md and report.html

# optional AI narrative
export ANTHROPIC_API_KEY=your_key
python app.py sample_data.json --ai
```

The repository contains **app.py**, **sample_data.json** and **README.md** (supplied alongside this PDF). The full source of app.py is reproduced in the appendix so this document is self-contained.

Note for submission: create a public GitHub repository, upload the three files, and paste the repository link into the Google Drive submission folder.

Appendix A: Methodology Notes and Submission Checklist

A.1 Planning Methodology

- **Hybrid:** upfront charter, WBS, risk register and phase gates (waterfall-style governance) with Scrum execution in 2-week sprints. This gives leadership date and cost predictability while keeping engineering flexible.
- **Estimation:** story points for sprint planning, person-weeks for WBS; velocity is measured from Sprint 1 and the plan is re-forecast each sprint.
- **Controls:** weekly risk review, change control board, Definition of Done, quality gates at Alpha, Beta, code freeze and go-live.
- **Early de-risking:** the two highest-uncertainty items (ASR/diarization quality and meeting-bot joining) are spiked in Sprint 1, and OAuth app reviews are submitted in Week 1.

A.2 Key Assumptions Summary

- Budget, velocity and utilization numbers are illustrative; actuals will replace them from Sprint 1.
- Vendors (meeting bot, ASR, LLM) are bought, not built. English only in MVP.
- No team size change except a 50% DevOps contractor and an external penetration test.

A.3 Submission Checklist

Deliverable	Status
Project Charter, WBS, Roadmap, Sprint Plan, Risk Register, Communication Plan, Resource Plan, Dashboard Mock-up, Architecture	Included in this PDF
AI Productivity Utility source code and README	Provided as separate files and appendix B; upload to GitHub
GitHub repository link	To be added by candidate
10 to 15 minute walkthrough video (Loom / YouTube unlisted / Drive)	To be recorded by candidate
Single Google Drive folder with all files	To be assembled by candidate

A.4 Suggested Walkthrough Video Outline (12 minutes)

Minutes	Topic
0:00-1:00	Approach, assumptions and how AI tools were used
1:00-3:00	Charter, scope and WBS
3:00-5:30	Roadmap, sprint plan and dependencies
5:30-7:00	Risk register and mitigation strategy
7:00-8:00	Communication plan, resource plan and dashboard
8:00-9:00	Change management scenario
9:00-10:00	Architecture walk-through
10:00-12:00	Live demo of the utility, challenges and learnings

Appendix B: Source Code of the AI Productivity Utility (app.py)

```
#!/usr/bin/env python3
"""PSI Sprint Status Reporter
Turns raw sprint metrics (JSON) into a leadership-ready weekly status report
(Markdown + HTML). Health (RAG) is computed from explicit, transparent rules.
Optional: add an AI-written executive narrative with the Claude API.

Usage:
    python app.py sample_data.json                # writes report.md + report.html
    python app.py sample_data.json --ai           # adds AI narrative (needs ANTHROPIC_API_KEY)
"""
import argparse
import html
import json
import os
import sys
import urllib.request


def compute_health(d):
    """Return (status, reasons). Rules are deliberately simple and explainable."""
    s = d["sprint"]
    expected = s["committed_points"] * s["day"] / s["total_days"]
    pace = s["completed_points"] / expected if expected else 1.0
    reasons, score = [], 0
    if pace < 0.7:
        score += 2; reasons.append(f"Delivery pace is {pace:.0%} of plan (red < 70%)")
    elif pace < 0.9:
        score += 1; reasons.append(f"Delivery pace is {pace:.0%} of plan (amber < 90%)")
    high = [r for r in d["risks"] if r["severity"] >= 15 and r["status"] == "Open"]
    if len(high) >= 3:
        score += 2; reasons.append(f"{len(high)} open high-severity risks")
    elif high:
        score += 1; reasons.append(f"{len(high)} open high-severity risk(s)")
    crit = d["bugs"].get("critical_open", 0)
    if crit:
        score += 2; reasons.append(f"{crit} open critical bug(s)")
    blockers = [b for b in d["blockers"] if not b.get("resolved")]
    if len(blockers) >= 3:
        score += 1; reasons.append(f"{len(blockers)} unresolved blockers")
    late = [m for m in d["milestones"] if m["status"] == "At Risk"]
    if late:
        score += 1; reasons.append(f"{len(late)} milestone(s) at risk")
    status = "RED" if score >= 4 else "AMBER" if score >= 2 else "GREEN"
    return status, reasons or ["All indicators within thresholds"], pace


def ai_narrative(d, status, reasons):
    key = os.environ.get("ANTHROPIC_API_KEY")
    if not key:
        return "(AI narrative skipped: set ANTHROPIC_API_KEY to enable.)"
    prompt = (
        "You are a technical project manager. Write a 90-word executive summary for "
        "leadership from this sprint data. Be factual; end with one clear ask.\n\n"
        f"Health: {status}. Reasons: {reasons}\nData: {json.dumps(d)}"
    )
    body = json.dumps({
        "model": os.environ.get("ANTHROPIC_MODEL", "claude-sonnet-5-5"),
        "max_tokens": 400,
        "messages": [{"role": "user", "content": prompt}],
    }).encode()
    req = urllib.request.Request(
        "https://api.anthropic.com/v1/messages", data=body,
        headers={"x-api-key": key, "anthropic-version": "2023-06-01",
                 "content-type": "application/json"})
    try:
        with urllib.request.urlopen(req, timeout=60) as r:
            return json.load(r)["content"][0]["text"]
    except Exception as e:  # keep the tool usable offline
        return f"(AI narrative failed: {e})"


def build_markdown(d, status, reasons, pace, narrative):
    s, b = d["sprint"], d["bugs"]
    L = [f"# Weekly Status Report: {d['project']}",
         f"**Sprint {s['number']}**  |  Day {s['day']} of {s['total_days']}  |  Overall health: **{status}**", ""]
    if narrative:
        L += ["## Executive summary", narrative, ""]
    L += ["## Why this status", *[f"- {r}" for r in reasons], "",
          "## Sprint progress",
          f"- Committed: {s['committed_points']} SP | Completed: {s['completed_points']} SP | Pace vs plan: {pace:.0%}",
          f"- Velocity (last sprints): {', '.join(map(str, d['velocity_history']))}", "",
          "## Quality",
          f"- Open bugs: {b['open']} (critical {b['critical_open']}) | Closed this week: {b['closed_this_week']}", "",
          "## Milestones", "| Milestone | Date | Status |", "|---|---|---|"]
    L += [f"| {m['name']} | {m['date']} | {m['status']} |" for m in d["milestones"]]
    L += ["", "## Top risks", "| Risk | Severity | Owner | Mitigation |", "|---|---|---|---|"]
    L += [f"| {r['name']} | {r['severity']} | {r['owner']} | {r['mitigation']} |"
          for r in sorted(d["risks"], key=lambda r: -r["severity"])[:5] if r["status"] == "Open"]
```

```
    L += ["", "## Blockers"]
    L += [f"- {x['text']} (owner: {x['owner']})" for x in d["blockers"] if not x.get("resolved")] or ["- None"]
    L += ["", "## Budget", f"- Spent {d['budget']['spent_pct']}% vs {d['budget']['planned_pct']}% planned"]
    L += ["", "## Decisions needed", *[f"- {x}" for x in d.get("decisions_needed", [])]]
    return "\n".join(L)


def md_to_html(md, status):
    colour = {"GREEN": "#1b8a3b", "AMBER": "#d98a00", "RED": "#c62828"}[status]
    out, in_ul, in_tbl = [], False, False
    for line in md.split("\n"):
        e = html.escape(line)
        e = e.replace("**", "\u0001")
        parts = e.split("\u0001")
        e = "".join(p if i % 2 == 0 else f"<b>{p}</b>" for i, p in enumerate(parts))
        if in_ul and not line.startswith("- "):
            out.append("</ul>"); in_ul = False
        if in_tbl and not line.startswith("|"):
            out.append("</table>"); in_tbl = False
        if line.startswith("# "):
            out.append(f"<h1>{e[2:]}</h1>")
        elif line.startswith("## "):
            out.append(f"<h2>{e[3:]}</h2>")
        elif line.startswith("- "):
            if not in_ul:
                out.append("<ul>"); in_ul = True
            out.append(f"<li>{e[2:]}</li>")
        elif line.startswith("|"):
            if set(line.replace("|", "").strip()) <= {"-"}:
                continue
            cells = [c.strip() for c in e.strip("|").split("|")]
            if not in_tbl:
                out.append("<table>"); in_tbl = True
                out.append("<tr>" + "".join(f"<th>{c}</th>" for c in cells) + "</tr>")
            else:
                out.append("<tr>" + "".join(f"<td>{c}</td>" for c in cells) + "</tr>")
        elif line.strip():
            out.append(f"<p>{e}</p>")
    if in_ul: out.append("</ul>")
    if in_tbl: out.append("</table>")
    css = (f"body{{font-family:Arial,sans-serif;max-width:860px;margin:30px auto;color:#222}}"
           f"h1{{border-bottom:4px solid {colour}}}table{{border-collapse:collapse;width:100%}}"
           f"td,th{{border:1px solid #ccc;padding:6px;text-align:left;font-size:13px}}th{{background:#f0f0f0}}")
    return f"<html><head><meta charset='utf-8'><style>{css}</style></head><body>{''.join(out)}</body></html>"


def main():
    ap = argparse.ArgumentParser(description="Generate a weekly sprint status report")
    ap.add_argument("data", help="path to sprint JSON")
    ap.add_argument("--ai", action="store_true", help="add AI executive narrative")
    ap.add_argument("--out", default="report", help="output file prefix")
    a = ap.parse_args()
    with open(a.data) as f:
        d = json.load(f)
    status, reasons, pace = compute_health(d)
    narrative = ai_narrative(d, status, reasons) if a.ai else None
    md = build_markdown(d, status, reasons, pace, narrative)
    with open(a.out + ".md", "w") as f:
        f.write(md)
    with open(a.out + ".html", "w") as f:
        f.write(md_to_html(md, status))
    print(f"Health: {status} | wrote {a.out}.md and {a.out}.html")


if __name__ == "__main__":
    sys.exit(main())
```